“板凳龙”运动轨迹模型优化研究

摘要

板凳龙作为闽南传统民俗，广受喜爱，用于迎神祈福。本文探究其在多种运动情境下的表现，分析其运动轨迹、速度变化、位置分布及碰撞检测。主要深入剖析复杂动态关键要素，优化得出运动轨迹，对其机理进行分析。

对于问题一：首先结合龙头初始状态，构建等距**螺线轨迹方程**$r(\theta)=8.8+\frac{0.55}{2\pi}\cdot\theta$。通过**步进法**模拟每秒龙头移动，推导出龙身、龙尾各节位置，并计算其在径向和轴向上的速度分量。对时间离散化处理，求得龙头至龙尾各部位在 0s 至 300s 每隔 60s 的位置与速度，并将其结果写入文件 result1.xlsx。

对于问题二：基于等距螺线轨迹方程，推导出**龙头位置坐标方程**，并计算龙头与龙身各节的最小距离。通过**几何与数值结合的方法**确定碰撞临界点及舞龙队盘入**中心点的终止时刻为 412.8s,最小距离**$d_{\min}=0.31325\,\mathrm{m}$，该条件在后续问题中会被反复使用。最终，计算该时刻舞龙队各部分的位置和速度，得到终止时刻及对应的位置数据和速度数据，并写入到文件 result2.xlsx 中。

对于问题三：分析了掉头过程中的约束条件，以螺距为优化变量，采用**遗传算法**对螺距进行优化求解，找到满足所有约束条件包括**最小碰撞距离**$d_{\min}=0.31325\,\mathbf{m}$与板凳宽度$d_{width}=0.15\,\mathrm{m}$的加和以及**调头区域半径**$r=4.5\,\mathbf{m}$，构建目标函数求得**最小螺距**$k_{\min}=0.5288\,\mathbf{m}$，通过对**时间 t 迭代**，评估不同螺距下的掉头角度。根据优化目标选择最优螺距，并验证其满足所有约束条件。

对于问题四：构建了调头圆弧的几何模型，发现**无法在同时满足相切与最短路径条件下缩短掉头曲线**。结合龙头恒定速度，通过板凳在螺线与圆弧、圆弧与圆弧之间的关系计算舞龙队各部分的位置与速度变化，并给出了不同时间点的数据。

对于问题五：根据问题四中的**路径和速度限制条件**建立数学模型。然后以龙头**行进速度为优化变量**，以舞龙队各把手速度不超过 2m/s 为约束条件建立优化问题，通过**数值差分法**求解龙头的最大行进速度。

关键词：等距螺线方程 仿真模拟 碰撞检测 遗传算法 数值差分法

一、问题重述

1.1 问题背景

板凳龙主要流行于闽南地区，是人们为了迎接神，祈求福音的一种传统民俗活动。每年元宵节人们将一条条板凳首尾相连，连接成蜿蜒曲折的形似龙一样的图腾标志。在盘龙的过程中，龙头在前领头、龙身和龙尾紧随其后，可达两百多米，非常壮观。板凳龙示意简图如图 1.1.1 所示

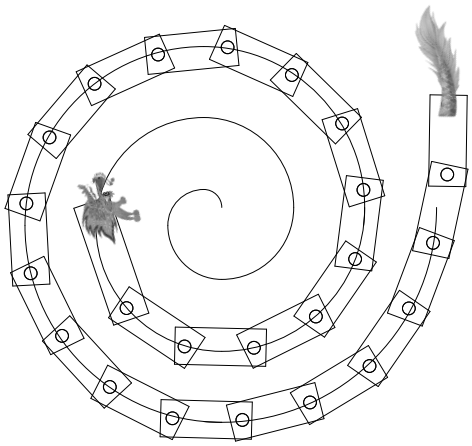

图 1.1.1 板凳龙示意简图

1.2 问题要求

现规定某舞龙队板凳龙由 223 节板凳组成，其中第一节为龙头，后面 221 节为龙身，最后一节为龙尾。板凳板长为 341cm，龙身和龙尾的板凳长均为 220cm，每个板凳均宽 30cm。每节板凳上都有两个直径为 5.5cm 的小孔，且每个小孔的中心距其最近板头 27.5cm，每个小孔均位于螺线轨迹上。

问题一：在本问题中，舞龙队运动轨迹沿螺距为 55cm 的螺线顺时针盘入，运动过程中速度始终保持 1m/s，龙头初始极坐标为$(8.8,0)$（单位：m），要求从初始时刻运动至 300s 时整体停止运动。求从初始时刻到 300s 时，龙头、龙身和龙尾各前把手及其后把手中心的位置和速度。并且给出在龙头前把手和后第 1、51、101、151 和 201 节龙身前把手和龙尾后把手在 0s，60s，120s，180s，240s，300s 这时刻的坐标和在该时刻的速度。

问题二：在本问题中舞龙队按照问题一中的速度和初始位置，沿问题一的运动路径螺线盘入。现要求在五板凳发生碰撞的前提下，舞龙队盘入到中心点的终止时刻，并给出该时刻舞龙队的位置和速度，以及与此同时龙头前把手、龙头后第 1、51、101、151、

201 调龙身前把手和龙尾后把手的位置和速度。

问题三：在本问题中舞龙队的运动路径、速度、初始位置均同问题一一致。现要求舞龙队在到达终止位置时掉头，然后逆时针盘出，掉头空间为以螺线中心为圆心、直径为 9m 的圆形区域，确定在使得龙头前把手能够沿着相应的螺线盘入到掉头空间的边界下的最小螺距。

问题四：在本问题中舞龙队盘入螺线的螺距为 1.7m，盘出螺线与盘入螺线关于螺线中心呈中心对称，舞龙队在问题 3 设定的调头空间内完成路径由两端圆弧相切连接而成的拟 S 曲线（前半段圆弧半径是后半段半径的 2 倍）。现要求保持各部分相切的条件下，最短的掉头曲线。并且在以调头开始时间为 0 的时刻，给出从-100s 到 100s 过程中，每秒舞龙队的位置和速度。

问题五：本问题要求按照问题四的路径，龙头行进速度保持不变，且舞龙队各把手的速度不超过 2m/s 的情况下，龙头的最大行进速度。

二、问题分析

2.1 问题一分析

（1）**轨迹参数求解与运动模拟：**已知舞龙队遵循等距螺线的运动轨迹，并给定了其运动速度及起始坐标。为了精确模拟其运动路径，首先对等距螺线的轨迹参数进行求解，进而建立起极坐标下的轨迹方程。由于舞龙队速度恒定，可采用步进法，通过每秒计算龙头在螺线上移动的具体距离，来构建运动学方程。通过螺线的极坐标方程，使用迭代法对数据进行计算，得到龙头、龙身、龙尾的位置坐标及速度。

（2）**速度分布与运动状态描述：**龙头以恒定的 1m/s 速度前进，而龙身和龙尾上各点的速度则不超过这一线速度。由于板凳龙整体在螺线上连续延伸，龙身和龙尾上的每一点都会随着它们在螺线轨迹上的位置不同而经历变化的速度状态。为了准确地模拟板凳龙上各点的运动状态，需要分别计算这些点在径向和周向上的速度分量，从而得出舞龙队在不同坐标下的速度分布。

2.2 问题二分析

（1）**深入轨迹模型分析与螺线方程的应用：**由问题一的基础信息出发，舞龙队的行进路径可由等距螺线来拟合。通过这一特性，可以推导出描述全程轨迹的螺线方程，进一步推导出龙头在任意时刻的位置坐标方程。进而，通过考虑板凳龙的整体结构，即龙头、紧随其后的龙身以及收尾的龙尾，可以将龙头的轨迹方程推广至整支舞龙队，从

而推导出龙身各节及龙尾在不同时刻的精确位置坐标方程。

（2）**优化几何模型推导策略以避免碰撞：**为模拟舞龙队的整体运动过程并有效避免碰撞，可以采用几何模型推导的方法进行优化。首先将板凳龙构造成一个几何模型，该模型反映龙头、龙身及龙尾之间的相对位置与尺寸关系。接着，利用这一模型来模拟舞龙队在螺线上的动态行进过程，特别是关注龙头与龙身之间的相对运动状态。在模拟过程中，通过计算和分析龙头与龙身各节之间的几何关系，可以准确找出可能导致碰撞的临界点，计算出临界时刻。

2.3 问题三分析

（1）调头螺距约束条件分析：首先，针对舞龙队在以中心为原点、具有固定半径的圆周上进行的调头动作，需要识别并分析其运动过程中的最大及最小约束条件。这一最大约束条件主要涉及到队伍在圆周上的物理运动限制，如保持固定的运动半径、确保团队成员之间的间距以防止相互碰撞。问题二中关于碰撞临界点的讨论应推广至本问，需判断是否为最小约束条件。

（2）调头最小螺距优化分析：首先需要明确优化问题的约束条件和目标函数，可以采用时间与角度之间的互相转换，以此来对应时间与螺距之间的关系，然后使用遗传算法对角度进行大量数值离散处理，以此拟合螺线运动轨迹，以及求出最小螺距。

2.4 问题四分析

（1）优化掉头圆弧的策略：优化掉头圆弧的关键在于在保持两段圆弧相切且半径比为 2:1 的约束条件下，通过调整圆弧半径来最小化掉头曲线的总长度。首先建立一个几何模型，以明确各段圆弧与螺线之间的相切关系，并推导出各段圆弧半径与掉头曲线总长度之间的数学表达式。随后，利用数值分析法求解问题。

（2）掉头曲线存在下的板凳龙速度与位置求解：鉴于掉头曲线的存在，板凳龙在掉头空间内的运动状态呈现出其独特性不同与盘入盘出螺线的运动过程。在求解板凳龙在掉头过程中的速度与位置时，首先需要明确掉头曲线的具体形态，即将其参数化表达。随后，结合龙头速度恒定为 1m/s 的已知条件，可以利用参数方程和微积分方法来逐步推导板凳龙上各把手在掉头过程中的位置与速度。当两个把手分别处于掉头曲线的不同区间时，分别求解不同过渡区域的不同坐标求解方法，以确保计算的准确性和完整性，结合几何关系进而求解。

2.5 问题五分析

（1）运动特性对速度的影响：鉴于问题四已经明确了舞龙队行进的复杂路径，其中龙头的运动轨迹引领整个队伍动态变化。由于舞龙队中的各节板凳是紧密相连且依次跟随龙头运动的，因此龙头的速度变化将直接传导至后续的每一节板凳，形成一个动态的连锁反应。这一过程中，需要揭示并量化各节板凳角度与速率之间的内在关联。

（2）几何特性对速度的影响：舞龙队行进的路径充满了曲率变化，尤其是调头区域的 S 形曲线，其曲率和长度的变化将直接作用于运动中的板凳，导致速度发生显著变化。特别是当龙尾附近的板凳穿越曲率较大的区域时，其速度可能会显著超过龙头的速度，从而增加了控制难度和碰撞风险。因此，建立一个能够全面考虑路径的几何特性对速度的影响，分析每一节板凳在行进过程中的速度变化，特别是龙尾附近板凳在通过高曲率区域时的速度表现，并且确保各部分的速度均不超过预设的安全阈值。

三、模型假设

- 假设舞龙队运动轨迹足够光滑，完全拟合等距螺线；
- 假设板凳长度和宽度固定，不因受力而变形；
- 假设速度计算中忽略空气阻力，地面摩擦力等外界因素。

四、符号说明

符号	含义
v	线速度
ω	角速度
θ	角度
$\left(r\left(\theta_t\right),\theta_t\right)$	极坐标下处于 t 时刻的位置
r	极径
k	螺距

五、问题一模型建立与求解

5.1 初始位置和角速度的设定

龙头前端的初始位置在极坐标系中明确为$(8.8,0)$，其极坐标与龙头把手入场时所需遵循的直线行进路径的起点相重合，因此不论是以前端作为起始还是以龙头前把手作为

起始点均合理。其中具体情境可参考图 4.1.1：

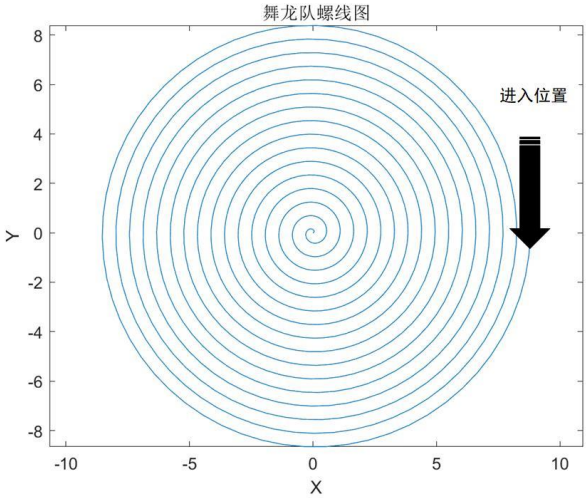

图 5.1.1 路径进入示意图

因此初始半径 $r_0 = 8.8m$，初始角度 $\theta_0 = 0$，龙头初始坐标为 $(8.8,0)$。

角速度 ω 是通过龙头的线速度 v_0 和 r_0 来计算，以此确保龙头的速度始终为 1m/s，其中角速度计算公式为 $\omega = \frac{v}{r}$。将龙头的线速度 v_0=1m/s 和龙头当前所在位置的极径 r =8.8m 代入公式，即可求得龙头的初始角速度 $\omega_0 = 0.11364$。

5.2 前 300 秒舞龙队轨迹模型拟合

由于龙头的线速度恒定为 1m/s，因此能够将螺线上的圆周运动视为一种与直线运动相结合的复合运动形式。所以，螺线的每一段都可以被看作是一个周期性的直线运动的集合，其中每一个周期，龙头都会沿着螺线在径向方向上前进相同的距离。

因为螺距为 k 始终不变，且每圈半径增加量为 $\frac{k}{2\pi}$，由此可以求得等距螺线方程为：

$$r(\theta) = r_0 + \frac{k}{2\pi} \cdot \theta \tag{1}$$

其中，$t \in [0,300]$（单位：秒），初始半径 $r_0 = 8.8$ 米，螺距 $k = 0.55$ 米。

5.3 板凳把手的角度差计算

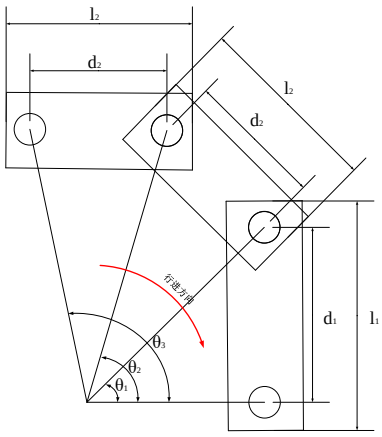

图 5.3.1 偏移角示意图

龙头前后把手、龙身和龙尾之间的角度差通过两者之间的实际弧长和当前半径的比值来计算，计算公式如下：

$$\Delta\theta_L = \frac{\Delta L - 2\cdot\varphi}{r} \tag{2}$$

其中 $\Delta\theta_L$ 为角度差，ΔL 为龙头与龙身（或龙身与龙身，龙身与龙尾）两把手之间的实际距离，φ 为偏移量，其中将 ΔL 近似为圆弧长度的误差可忽略不计。

5.4 建立龙头坐标方程

根据极坐标转换为直角坐标公式，公式如下：

$$\begin{cases} x = r\cdot\cos\theta \\ y = r\cdot\sin\theta \end{cases} \tag{3}$$

由于 θ 无法消去，因此等距螺线的直角坐标方程为以 θ 为参数的参数方程，方程如下：

$$\begin{cases} x(\theta) = \left(r_0 + \dfrac{k}{2\pi}\theta\right)\cos\theta \\ y(\theta) = \left(r_0 + \dfrac{k}{2\pi}\theta\right)\sin\theta \end{cases} \tag{4}$$

其中 $r_0 = 8.8$ 米，螺距 $k = 0.55$ 米。

已知龙头位置速度恒定为 1m/s，由角速度公式 $\omega = \dfrac{\theta}{t}$，可以得出螺线的极角 θ 与时间 t 的关系为：

$$\theta(t) = \frac{2\pi}{k}\cdot v\cdot t \tag{5}$$

其中 $v = 1\text{m/s}$ 。

因此可以将$\theta-t$关系带入螺线方程，得到龙头在任意时刻t时的位置方程：

$$r(t)=r_0+vt \tag{6}$$

将螺线的极坐标转化为笛卡尔坐标，因此可以转换龙头在时间t的直角坐标的参数方程为：

$$\begin{cases} x(t)=r(t)\cdot\cos(\theta(t))=(r_0+v\cdot t)\cdot\cos\left(\dfrac{2\pi}{k}\cdot v\cdot t\right) \\ y(t)=r(t)\cdot\sin(\theta(t))=(r_0+v\cdot t)\cdot\sin\left(\dfrac{2\pi}{k}\cdot v\cdot t\right) \end{cases} \tag{7}$$

5.5 建立龙身坐标方程

对于龙身和龙尾的每一节板凳，它们以固定距离跟随龙头，现定义龙身的两把手之间长度为L，则第n节板凳距离龙头的偏移为$n\cdot L$，因此对于第 n 节板凳，其相对于龙头的角度偏移为：

$$\Delta\theta_n=\frac{n\cdot L}{r(t)} \tag{8}$$

其中以两把手之间长度L代替弧长的误差可忽略不计。

因此由 5.4 节龙头坐标方程可得第 n 节板凳的参数方程为：

$$\begin{cases} x_n(t)=r(t)\cdot\cos(\theta(t)-\Delta\theta_n) \\ y_n(t)=r(t)\cdot\sin(\theta(t)-\Delta\theta_n) \end{cases} \tag{9}$$

其中$\theta(t)=\dfrac{2\pi}{k}\cdot v\cdot t$，$\Delta\theta_n=\dfrac{n\cdot L}{r(t)}$，$r(t)=r_0+vt$，该参数方程可以描述龙身、龙尾（第$n$节板凳）随时间变化的任意坐标。

5.6 坐标求解

根据 5.4 节和 5.5 节的坐标公式，使用 Matlab 对自变量t以 1s 为单位对时间离散化处理进行数值模拟，得到在 0 s、60 s、120 s、180 s、240 s、300 s 时，龙头前把手、龙头后面第 1、51、101、151、201 节龙身前把手和龙尾后把手的位置，见附录表 1.1。

5.7 速度方程建立和求解

每节板凳的速度是根据两次位置之间的位移变化进行微分计算的，公式如下：

$$v=\frac{\sqrt{(x_L-x_B)^2+(y_L-y_B)^2}}{dt} \tag{10}$$

其中前位置坐标(x_L, y_L)和后位置坐标(x_B, y_B)可根据龙头（或龙身）坐标方程求出。

对自变量t以 1s 为单位对时间离散化处理，不断进行数值模拟，可求得在 0 s、60 s、120 s、180 s、240 s、300 s 时，龙头前把手、龙头后面第 1、51、101、151、201 节龙身前把手和龙尾后把手的速度数据，见附录 1.2。

六、问题二模型建立与求解

6.1 舞龙队全程轨迹模型拟合

由第一问可知，舞龙队运动轨迹符合$r_0 = 8.8$，$k = 0.55$的等距螺线，因此轨迹方程为：

$$r(\theta) = r_0 + \frac{k}{2\pi} \cdot \theta \tag{11}$$

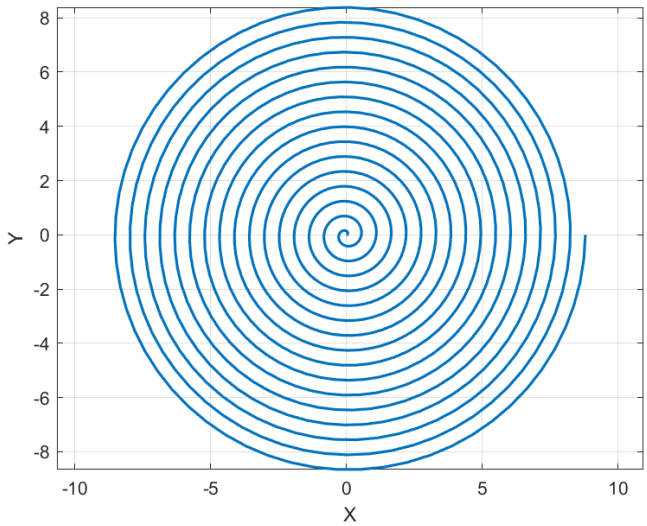

图 6.1.1 舞龙队全程等距螺线轨迹图

6.2 基于几何模型的碰撞临界值判定

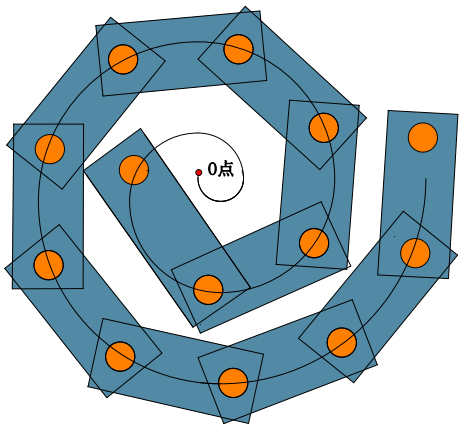

图 6.2.1 碰撞模拟示意图

在几何模型中，板凳可以抽象为一个矩形，每个矩形由其中心点，长度（龙身长 2.2 米），宽度（0.3 米）以及旋转角进行定义，如图 6.2.1 所示。假设第 n 节板凳的中心位置为 (x_n, y_n)，旋转角度为 θ_n，则第 $n+1$ 节板凳的中心位置为 (x_{n+1}, y_{n+1})，旋转角度为 θ_{n+1}。

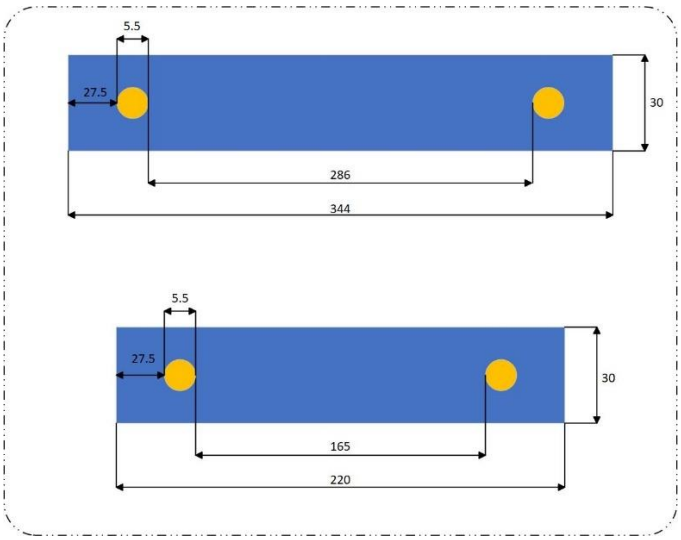

图 6.2.2 板凳几何抽象示意图

因此每个板凳的四个顶点可以根据板凳中心点和旋转角度通过几何关系变换计算出来：

$$P_1 = \left(-\frac{L}{2}, -\frac{W}{2}\right) \quad P_2 = \left(-\frac{L}{2}, \frac{W}{2}\right)$$
$$P_3 = \left(\frac{L}{2}, \frac{W}{2}\right) \quad P_4 = \left(\frac{L}{2}, -\frac{W}{2}\right) \tag{12}$$

其中 $P_1 \ldots P_4$ 为板凳相对于中心的四个顶点 L 为每个板凳的长度，W 为每个板凳的宽度。

已知板凳具有一个旋转角度 θ，则顶点的旋转矩阵为：

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \tag{13}$$

将每个顶点进行旋转和位移变换可得到实际坐标方程为：

$$P_i' = R(\theta) \cdot P_i + (x_n, y_n) \tag{14}$$

对于两节相邻板凳，可以计算其四个顶点之间的最小距离，也就是计算第 n 条板凳的一条边，定义为通过两个顶点的向量 $\vec{e} = P_{n+1} - P_{n+1}'$，其中点到直线距离公式为

$d=\frac{|\vec{e}\times\vec{p}|}{|\vec{e}|}$。在公式（14）中 $\vec{p}$ 是从顶点 P_i 到边上任意一点的向量，$\times$表示叉积，$|\vec{e}|$ 是边的长度。因此可以计算所有顶点到另一块板凳的边的距离，存在最小距离 $d_{\min}=0.31325$ 为把手到相邻最近龙身的距离。

图 6.2.3 为碰撞区间模拟，实线部分为龙身与龙头未碰撞位置，虚线部分为龙身与龙头碰撞位置。

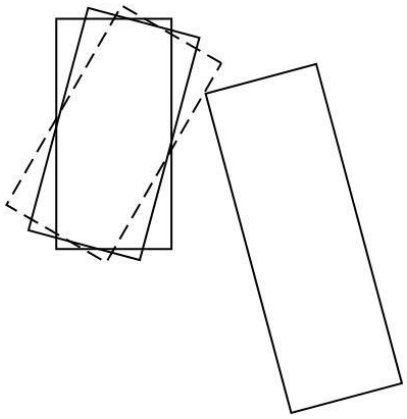

图 6.2.3 碰撞区间模拟示意图

所以当龙头的顶点不在龙身的运动区域内时，则认为不发生碰撞。

6.3 模型求解

对于上述模型，根据代码求解可得在 412.8s 处，前把手距离外龙身最近，此时龙头与龙身发生碰撞，此时龙头的坐标为$(1.1835,1.2250)$，此时的速度为 1.2659m/s，整个舞龙队此刻的位置和速度详见 result2.xlsx。除此之外，此刻龙头前把手、龙头后第 1、51、101、151、201 条龙身前把手和龙尾后把手的坐标和速度见附录 1.3。

七、问题三模型建立与求解

7.1 建立舞龙队盘入轨迹方程

由于舞龙队在调头前轨迹仍拟合等距螺线，因此此时的等距螺线方程为：

$$r(\theta)=r_0+\frac{k}{2\pi}\cdot\theta \tag{15}$$

由于本问要求对螺距进行优化，所以假定初始状态时最外层至最里层圈数为 16，螺距为 k_0，因此极径 $r_0=16k_0$。

7.2 调头区域约束条件分析

由题干可知调头区域是以螺线中心为圆心，9m 为直径的圆形区域，如图 7.2.1 所示：

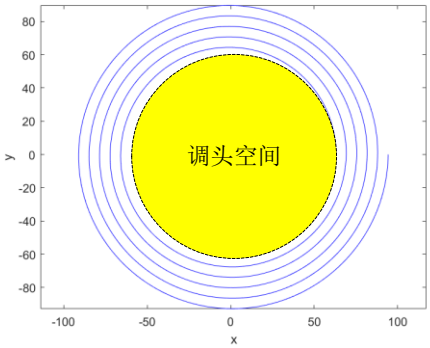

图 7.2.1 调头区域图

舞龙队调头进入区域的半径应为 $r_h = 4.5\,\mathrm{m}$，因此存在螺距 k_0，使得在某一时刻 θ 满足 $r(\theta) = r_0 + \frac{k_0}{2\pi}\theta = 4.5$，假设此时的角度为 θ_{off}，因此可以转换为：

$$\theta_{off} = \frac{2\pi(4.5 - r_0)}{k_0} \tag{16}$$

因为角速度公式为：$\begin{cases} \omega = \dfrac{\theta}{t} \\ \omega = \dfrac{v}{r} \end{cases}$，所以根据上述条件，可以得到时间 t 关于螺距 k_0 的关系式：

$$t_{off} = \frac{\theta_{off} \cdot r(\theta_{off})}{v_0} = \frac{9\pi(4.5 - r_0)}{k_0 \cdot v_0} \tag{17}$$

其中 t_{off} 为调头时间，v_0 为 1m/s 的龙头线速度。

由问题二得碰撞临界条件为：当龙头前把手到临近龙身的垂直距离小于 0.31325m 时将产生碰撞。因此螺距存在最大下界，使得：

$$k_0 > d_{\min} + \frac{1}{2} d_{width} = 0.46325 \tag{18}$$

7.3 基于遗传算法对最小螺距的优化求解

GA 遗传算法（Genetic Algorithm）是一种基于自然选择和遗传学原理的搜索算法，它通过模拟生物进化过程中的选择、交叉和变异等机制，来迭代改进候选解决方案，适用于解决复杂优化问题。本题中遗传算法主要过程如下：

1、生成初始种群

对角度θ进行离散化处理，逐渐对角度θ_i递增直至等于θ_{off}。

$$\theta_i=\left[0,\frac{\theta_{off}}{n-1},\frac{2\theta_{off}}{n-1},\frac{3\theta_{off}}{n-1},\ldots,\frac{(n-1)\theta_{off}}{n-1}\right],\ i=1,2,3,\ldots,n-1 \tag{19}$$

由 7.2 节中公式（21）可知，$t-\theta$的关系为：

$$t_i=\frac{\theta_i\cdot r(\theta_i)}{v_0} \tag{20}$$

在满足约束条件$t_i>0$的情况下，选择了 1000 个θ转换为时间t，作为初始种群数。

2、个体适应度评价

适应度函数是 GA 遗传算法中重要的评价指标，一般情况下为优化求解的目标函数。本文中要求对螺距k进行优化，时间与螺距k之间的关系是复杂且相互依赖的。时间不是直接优化螺距k的目标，但螺距k的选择会显著影响舞龙队完成特定动作（如盘入、盘出、调头）所需的时间。由题干所知在舞龙队能够自如地盘入和盘出的前提下，盘龙所需要的面积越小、行进速度越快，则观赏性越好，因此在实际操作中，舞龙队往往希望能够在尽可能短的时间内完成复杂的动作，以提高观赏性和效率。因此时间最小化时一定存在一个最优的螺距k，该螺距在满足所有约束条件的同时，使舞龙队能够最快地完成动作。

结合 7.2 节中的约束条件，目标函数如下：

$$\begin{aligned}&\min t=\frac{9\pi(4.5-r_0)}{k_0\cdot v_0}\\&s.t.\begin{cases}k_0>0.46325\\ v_0=1\\ t_{off}>0\end{cases}\end{aligned} \tag{21}$$

3、物种选择与淘汰

选择与淘汰是产生最优个体的方式。通过对于目标函数在约束条件内的优化求解可以得出螺距k区间。使用 Matlab 不断通过θ对时间t进行数值转换，然后迭代模拟舞龙队的轨迹以及求出螺距k的最优区间为$[0.5288,0.5298]$，因此最小螺距$k_0=0.5288\,\mathrm{m}$。

八、问题四模型建立与求解

8.1 几何模型法对调头路径分析

已知调头区域内轨迹为两个相切圆弧，因此两个圆弧所在的两个圆也相切，两个圆与盘入、盘出螺线也均相切，设盘入、盘出点为A,C，作直线l_1，l_2切圆O_1，O_2于点C，A。假设小圆弧所在圆为O_1，大圆弧所在圆为O_2，因为调头轨迹呈中心对称，因此线段O_1O_2经过两圆的切点B，现经过点A向直线l_2作垂线交l_2于点D。连接O_1C，O_2A，AC，因为直线l_1，l_2切圆O_1，O_2于点C，A，因此O_1C，O_2A分别垂直于直线l_1，l_2，且O_1C平行于O_2A,所以A，O_2，D三点共线$\angle AO_2O_1=\angle CO_1O_2$，又因为$\angle ABO_2=\angle O_1CO_2$，所以$\triangle O_1CB\sim\triangle O_2AB$。设$\angle CAD$为$\alpha$，$\angle O_1O_2D$为$\beta$，过点$O_1$作线段$O_1G$垂直于$AD$。因此可将两圆弧抽象为平面中的圆来探讨路径问题，如图 8.1.1。

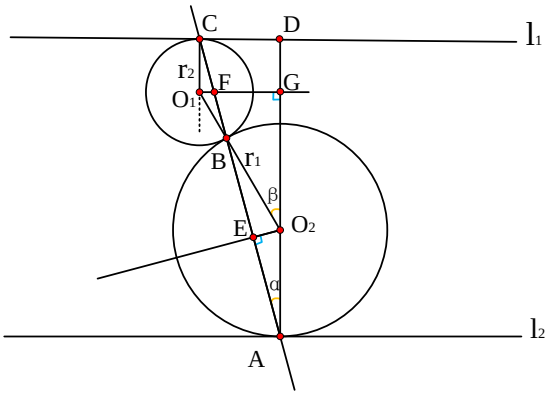

图 8.1.1 平面几何模型

要想求调头曲线的长度，只需求AB，BC的和，设AB，BC的长度为S_1，S_2，因此曲线长度为$S_t=S_1+S_2$ 。已知弧长公式为$S=r\cdot\theta$，其中r，θ分别为弧半径和弧度，已知$\angle AO_2B=\angle CO_1B=\pi-\beta$，因此$AB$和$BC$的长度为：

$$\begin{aligned}S_1&=r_1(\pi-\beta)\\S_2&=r_2(\pi-\beta)\end{aligned}\tag{22}$$

所以

$$S_t=(r_1+r_2)\cdot(\pi-\beta)\tag{23}$$

又因为$O_1G\parallel CD$，$O_1C\parallel GD$，所以四边形O_1CDG为平行四边形，所以$O_1G=CD$。因为$d_{CD}=2d\sin\alpha$，$d_{O_1G}=(r_1+r_2)\sin\beta$，所以$(r_1+r_2)\sin\beta=2d\sin\alpha$，因此

$$\beta = \arcsin \frac{2d\sin\alpha}{r_1 + r_2} \tag{24}$$

令 $t = r_1 + r_2$，所以总弧长为：

$$S_t = t \cdot \left(\pi - \arcsin \frac{2d\sin\alpha}{t} \right) \tag{25}$$

又因为

$$\begin{cases} 2d\cos\alpha = (r_1 + r_2) + (r_1 + r_2)\cos\beta = t + t\cos\beta \\ 2d\sin\alpha = (r_1 + r_2)\sin\beta = t\sin\beta \end{cases} \tag{26}$$

所以联立方程得 $t = \dfrac{d}{\cos\alpha}$。

因为 α 为切线 l_1，l_2 间距与调头区域圆直径的夹角，若使两个圆弧组成的路径始终与盘入、盘出螺线相切，则 α 角度不变，所以 t 是一个常量，因此路径总长 $S_t = t\left(\pi - \arcsin \dfrac{2d\sin\theta}{t} \right)$ 恒定不变。

因此不能在保持各部分相切的情况下调整圆弧使调头曲线变短。

8.2 相切条件判断

假设圆弧 O_2 在直角坐标系下表示为：

$$(x - x_2)^2 + (y - y_2)^2 = r_2^{\,2} \tag{27}$$

其中 (x_2, y_2) 为圆弧的圆心坐标，r_2 为圆弧的半径。

要找到螺线与圆弧的相切点必须同时满足两个条件：

1、位置重合条件：相切点必须同时位于螺线的方程和圆弧的方程上。这意味着该点的坐标需要同时满足螺线的参数方程。这一条件确保了相切点确实存在于螺线与圆弧的交界处。

2、切线斜率相等条件：在相切点处，螺线的切线斜率必须与圆弧的切线斜率完全相等。由于螺线是曲线，其切线斜率随位置变化而变化，因此需要通过求导得到螺线在相切点处的切线斜率。同样地，圆弧作为圆的一部分，其切线斜率可以通过圆心与相切点连线的垂直关系来确定。确保两者斜率相等，是判断两曲线在该点相切的关键几何依据。

8.3 建立盘入、盘出螺线轨迹微分方程求解龙各部位位置坐标

由于舞龙队在盘入调头和盘出区域，轨迹均拟合等距螺线，如图 8.3.1。

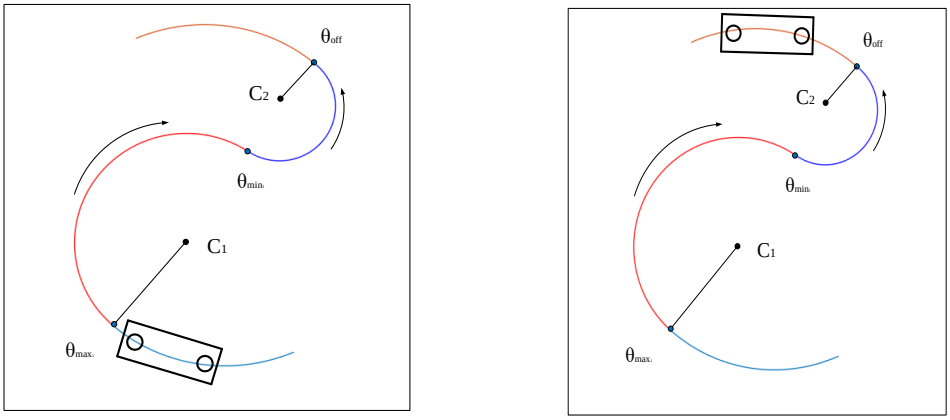

图 8.3.1 板凳完全位于盘入、盘出螺线位置图

但是螺距 k 变更为 1.7m，所以依旧假定龙头由距中心 16 圈处开始运动，轨迹方程为 $r(\theta)=r_0+\frac{k}{2\pi}\cdot\theta$，其中 $r_0=27.2\,\mathrm{m}$，螺距 $k=1.7\,\mathrm{m}$。

由问题三可知，调头位置为以螺线中心为圆心，半径 $d=4.5$ 的圆形区域，若要使得调头路线与盘入、盘出螺线均相切，则舞龙队进入调头区域的点即为调头路线与盘入、盘出螺线相切的点，如图 8.3.2 所示：

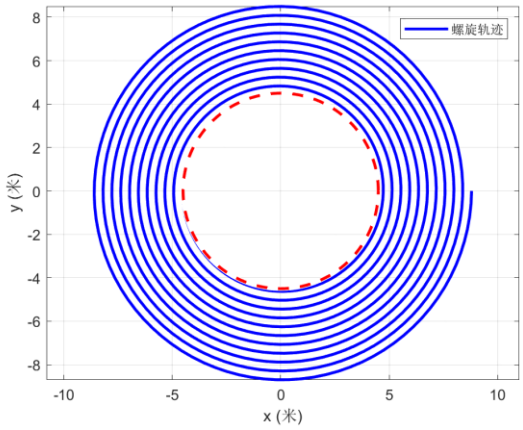

图 8.3.2 盘入路径与调头区域切入点

此刻龙头处于调头点的时间 $t=0$。根据问题三中求得的 $\theta-t$ 的关系，可以求得盘入螺线的 θ 关于 t 的常微分方程为：

$$\frac{d\theta}{dt}=\frac{1}{\varepsilon\sqrt{1+\theta^2}} \tag{28}$$

其中 ε 为切线斜率，计算公式为：

$$\varepsilon=\frac{\frac{dx}{d\theta}\sin\theta+r\cos\theta}{\frac{dr}{d\theta}\cos\theta-r\sin\theta}=\frac{a\sin\theta+\left(r_0+a\theta\right)\cos\theta}{a\cos\theta-\left(r_0+a\theta\right)\sin\theta} \tag{29}$$

将时间从-100 到 0 以 1s 为单位离散化，采用龙格库塔法对微分方程求解，当时间$t=0$时，龙头位置$\theta(0)=\theta_{in}$，此时的θ_{in}是螺线与调头区域圆周的切点对应的极角。此时在盘入螺线上龙头、龙身的位置坐标就可以求出。

已知盘入与盘出螺线关于螺线中心呈中心对称，因此通过坐标转换可以求出盘出螺线的极坐标方程为：

$$r(\theta)=r_0+\frac{k}{2\pi}(\theta-\pi) \tag{30}$$

8.4 盘入螺线与圆弧C_1切点前后把手位置判定

此时板凳横跨盘入螺线与圆弧C_1区域如图 8.4.1 所示：

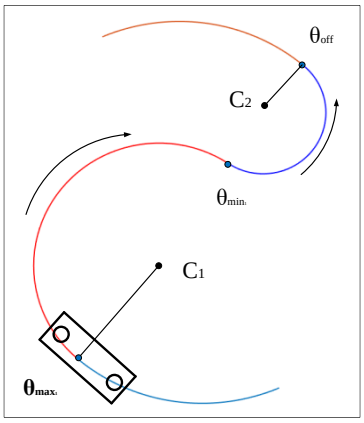

图 8.4.1 板凳横跨盘入螺线与圆弧C_1切点前后把手位置图

首先定义距离约束方程：

$$\left(k\theta\cos\theta-x_1\right)^2+\left(k\theta\sin\theta-y_1\right)^2=d^2 \tag{31}$$

其表示后把手与前把手之间的距离为d，需满足$\theta>\theta_1$（新点的位置必须大于已知点的角度）。所以推导出方程：

$$F(\theta)=\left(k\theta\cos\theta-x_1\right)^2+\left(k\theta\sin\theta-y_1\right)^2-d^2=0 \tag{32}$$

通过 Matlab 进行使用牛顿法求解后一把手位置坐标。

8.5 建立调头轨迹方程求解龙各部位位于调头轨迹上的位置坐标

舞龙队进入到调头区域后，开始以r_1为半径做线速度恒定为 1m/s 的圆周运

动，由于速度恒定，时间t与弧长s的关系时线性的，即$s=t$。弧长s与角度θ的关系由圆弧半径给出，即$s=r_1\times(\theta-\theta_{in})$。对于圆弧$C_1$，有方程$t=r_1(\theta-\theta_{\max_1})$，其中$\theta_{in}$为进入圆弧时的角度，因此可以求得极坐标下的圆弧$C_1$的轨迹方程：

$$\theta_{r_1}(t)=\frac{t}{r_1}+\theta_{\max_1} \tag{33}$$

将极坐标转换为直角坐标可得头节点在圆弧C_1上的位置坐标(x,y)，参数方程为：

$$\begin{cases} x_1(t)=r_1\cos(\theta_1(t))+x_1 \\ y_1(t)=r_1\sin(\theta_1(t))+y_1 \end{cases} \tag{34}$$

$(x_1,y_1)=(-0.76,-1.31)$是圆弧C_1的圆心坐标。

圆弧C_2的圆心坐标和方程同C_1类似，但是起始角度发生了偏移为$\theta_{\min}-\pi$。同样，时间t（从离开圆弧 C1 进入圆弧 C2 的时刻开始计时）与弧长s的关系是线性的。C_2的极坐标方程为：

$$\theta_2(t)=\frac{t}{r_2}+\theta_{\min_1}-\pi \tag{35}$$

直角坐标下的参数方程为：

$$\begin{cases} x_2(t)=r_2\cos(\theta_2(t))+x_2 \\ y_2(t)=r_2\sin(\theta_2(t))+y_2 \end{cases} \tag{36}$$

其中$(x_2,y_2)=(1.74,2.45)$为圆弧C_2的圆心坐标。

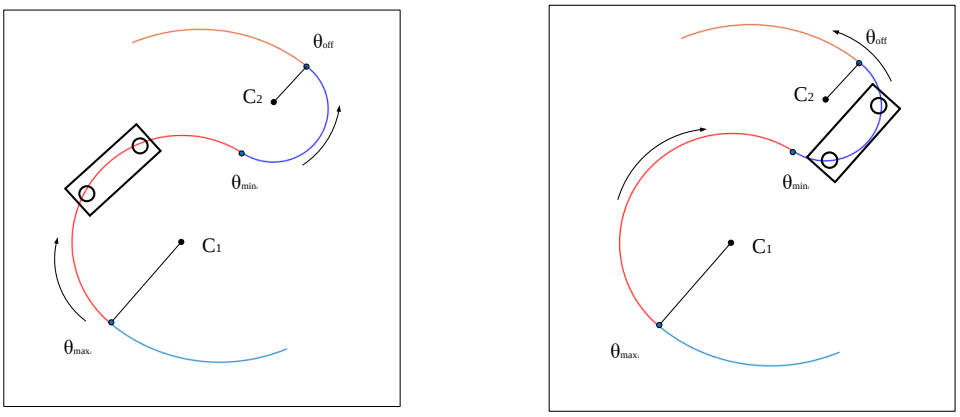

图 8.5.1 板凳完全位于圆弧C_1和圆弧C_2位置图

8.6 两圆弧切点前后把手位置判定及坐标求解

此时板凳横跨于圆弧C_1于圆弧C_2切点，如图 8.6.1 所示：

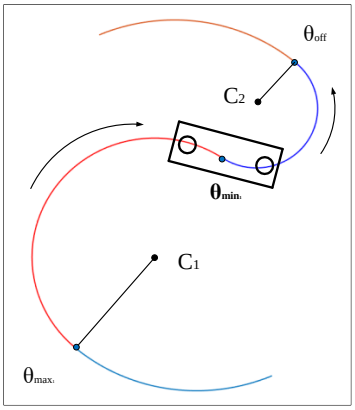

图 8.6.1 板凳横跨圆弧C_1与圆弧C_2切点前后把手位置图

已知前把手在圆弧C_2上的角度为θ_1，圆弧C_2的半径为r_2，两个相邻把手之间的距离为d。

角度增量计算：

$$\delta\theta = 2\arcsin\left(\frac{d}{2r_2}\right) \tag{37}$$

通过反正弦函数来计算由于增加距离d而在圆弧上产生的角度变化，如果$\delta\theta \le \theta_1 - \theta_{\min_2}$（$\theta_{\min_2}$是圆弧$C_2$起始角度），则下一把手仍在圆弧$C_2$上。则下一把手的角度和坐标公式为：

$$\begin{cases} \theta = \theta_1 - \delta\theta \\ x = x_2 + r_2\cos\theta \\ y = y_2 + r_2\sin\theta \end{cases} \tag{38}$$

如果此时$\delta\theta > \theta_1 - \theta_{\min_2}$，则此时下一个把手点从圆弧$C_1$过渡到圆弧$C_2$上，此时前把手到圆弧$C_1$的距离为$d_i = \sqrt{\left(x_1 - x_{C_1}\right)^2 + \left(y_1 - y_{C_1}\right)^2}$。由于把手点与圆弧$C_1$圆心连线，圆弧$C_1$半径和两把手见构成三角形，因此可以使用余弦定理计算角度增量：

$$\cos(\delta\theta)=\frac{d_i^2+r_1^2-d^2}{2d_i\cdot r_1}$$
$$\delta\theta=\arccos\left(\frac{d_i^2+r_1^2-d^2}{2d_i\cdot r_1}\right) \tag{39}$$

当前把手点与圆弧C_1圆心夹角为：

$$\theta_{C_1-d_i}=\arctan\left(\frac{y_1-y_{r_1}}{x_1-x_{r_1}}\right) \tag{40}$$

则下一把手点在圆弧C_1上的角度和坐标：

$$\begin{cases}\theta=\theta_{C_1_d_i}+\delta\theta\\ x=x_{c_1}+r_1\cos\theta\\ y=y_{c_1}+r_1\sin\theta\end{cases} \tag{41}$$

8.7 圆弧C_2与盘出螺线切点前后把手位置判定及坐标求解

此时板凳横跨于圆弧C_2与盘出螺线切点，如图 8.7.1 所示：

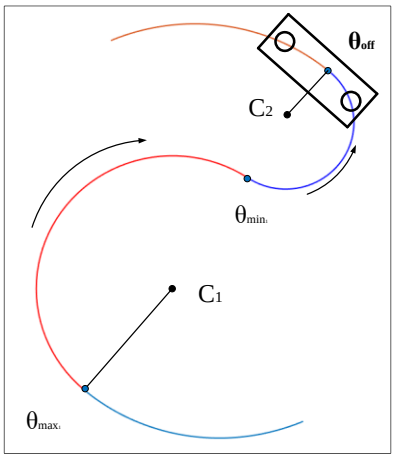

图 8.7.1 板凳横跨圆弧C_2与盘出螺线切点前后把手位置图

当下一把手也位于盘出螺线上时，角度$\theta\geq\theta_{\max_1}-\pi$，此时坐标计算公式推导同上述方法，位置参数方程为：

$$\begin{cases}x=k(\theta+\pi)\cos\theta\\ y=k(\theta+\pi)\sin\theta\end{cases} \tag{42}$$

当下一把手从圆弧C_2过渡到盘出螺线时，设下一个把手点在圆弧 C2 上的角度为θ_{next}，但由于该角度位置，所以定义一个关于未知角度ϕ的约束函数：

$$f(\phi)=\left[x_C+r_{C_2}\cos(\theta_{\max}-\phi)-x_1\right]^2+\left[y_C+r_{C_2}\sin(\theta_{\max}-\phi)-y_1\right]^2-d^2 \quad (43)$$

该函数表示从当前把手点到假设的圆弧C_2上某点的距离平方与d^2的差。使用数值方法求解方程$f(\phi)=0$，得到ϕ。因此下一个把手的角度和坐标方程为：

$$\begin{cases}\theta_{next}=\theta_{\max}-\phi\\ x=x_C+r_{C_2}\cos\theta_{next}\\ y=y_C+r_{C_2}\sin\theta_{next}\end{cases} \quad (44)$$

8.8 坐标求解

结合 8.3、8.4、8.5、8.6、8.7 节，可以求得在第−100 s、−50 s、0 s、50 s、100 s 时刻的龙头前把手、龙头后面第 1、51、101、151、201 节龙身前把手和龙尾后把手的位置坐标见附录 1.4。

8.9 求解龙各部位速度

设θ_n为第n个时间步长的角度值，由于已知盘入、盘出螺线微分方程以及圆弧方程，因此可以求得全程运动中的角度值θ，且可得知$\theta-t$的关系：

$$\frac{d\theta}{dt}=\frac{1}{k\sqrt{1+\theta^2}} \quad (45)$$

再由角速度公式可知$\omega=\dfrac{\theta_n-\theta_{n-1}}{\Delta t}$，因此差分速率求解法的公式如下：

$$V=-k\cdot\sqrt{1+\theta_n^2}\cdot\frac{\theta_n-\theta_{n-1}}{\frac{\Delta t}{2}} \quad (46)$$

由此−100 s、−50 s、0 s、50 s、100 s 时，龙头前把手、龙头后面第 1、51、101、151、201 节龙身前把手和龙尾后把手的速度可求解，见附件 1.5。

九、问题五模型建立与求解

9.1 数值差分法对速度求解并取得最大值

由第四问可知，轨迹方程和速度方程，由于此时速度v改变，因此需重新定义速度v与时间t的关系，已知前文中求得角度θ与速度v的关系，因此可将方程中角度对速度的求解映射为时间对速度的求解。

首先定义时间序列$t=\{t_1,t_2,\ldots,t_n\}$上某个把手（或龙头）在二维空间中的位置

由 $X(t)$ 和 $Y(t)$ 表示，其中 $X(t)$、$Y(t)$ 分别是 t 时刻该把手的 x 和 y 坐标组成的向量。

对于 $X(t)$ 的第一个时间点 t_1 和 $Y(t)$ 的第一个时间点，使用前向差分来计算速度：

$$\begin{aligned} v_x(t_1) &\approx \frac{X(t_2)-X(t_1)}{\Delta t} \\ v_y(t_1) &\approx \frac{Y(t_2)-Y(t_1)}{\Delta t} \end{aligned} \tag{47}$$

对于 $X(t)$ 和 $Y(t)$ 的中间时间点 $t_i\,(i=1,2,3,\ldots,n-1)$，使用中心差分来计算速度：

$$\begin{aligned} v_x(t_i) &\approx \frac{X(t_{i+1})-X(t_{i-1})}{2\Delta t} \\ v_y(t_i) &\approx \frac{Y(t_{i+1})-Y(t_{i-1})}{2\Delta t} \end{aligned} \tag{48}$$

其中上述两公式中 $\Delta t = t_2 - t_1$ 是时间步长。因此每个时间点上的和速度 $v(t)$：

$$v(t)=\sqrt{v_x(t)^2+v_y(t)^2} \tag{49}$$

使用 Matlab 在时间范围内对 v 进行运算并取得最大速度，最大速度 $v_{\max}=1.82\,\mathrm{m/s}$。

十、模型检测和灵敏度分析

由于螺距在等距螺线的极坐标方程中为自变量 θ 的系数，因此对螺距进行灵敏度分析灵敏度有着重要的意义。

采用局部灵敏度分析方法，即单参数灵敏度分析。在保持其他参数不变的情况下，分别设置不同的螺距值，观察舞龙队在不同螺距下的运动轨迹、速度变化及碰撞情况。通过比较不同螺距下的结果，评估螺距对盘龙表演的影响。

1、运动轨迹影响：随着螺距的增大，舞龙队的运动轨迹逐渐向外扩展，整体占地面积增加。螺距较小时，舞龙队能够更紧凑地盘旋，但可能因空间不足而增加碰撞风险；螺距较大时，虽然减少了碰撞风险，但可能因占地面积过大而影响观赏性。

2、速度变化影响：螺距的变化对舞龙队的速度分布也有显著影响。螺距较

小时，由于轨迹紧凑，龙身和龙尾部分的速度变化更加剧烈；螺距增大后，速度变化趋于平缓。合理的螺距选择可以使舞龙队的速度分布更加均匀，提升表演的观赏性。

3、碰撞情况分析：通过模拟不同螺距下的碰撞情况，发现螺距对碰撞风险有显著影响。螺距过小时，舞龙队内部容易发生碰撞；螺距适中时，碰撞风险较低；螺距过大时，虽然碰撞风险进一步降低，但可能因占地面积过大而增加外部碰撞的可能性。因此，需要选择一个既能保证安全性又能提升观赏性的螺距值。

通过灵敏度分析可知，螺距对“板凳龙”闹元宵表演的运动轨迹、速度变化及碰撞情况有显著影响。为了提升盘龙表演的观赏性和安全性，建议在实际表演中根据场地条件和表演需求选择合适的螺距值。通过多次模拟和实验验证，确定最优螺距范围，以确保舞龙队能够顺畅地盘旋并展现出最佳的表演效果。

十一、模型评价及推广

模型优点

1、本文构建了高度抽象的数学模型，捕捉了舞龙队运动的本质特征，通过等距螺线方程与数值差分法的巧妙结合，描绘了舞龙队的复杂运动轨迹，模拟了各部分的速度变化，确保模型在描述和预测上的准确性。

2、针对问题一和问题四，运用了常微分方程与数值差分方法，对舞龙队在不同时刻的位置坐标与速度进行精确的拟合，使得模型的更加有效准确。

3、为了求解调头路径中的最小螺距这一复杂优化问题，本文巧妙地引入了遗传算法。这一算法凭借其强大的全局搜索能力和高效的优化机制，极大地简化了求解过程，不仅找到了最优解，还显著提高了计算效率，展现了遗传算法在处理此类复杂优化问题中的独特优势。

模型缺点

1、本文中所有模型及结论都基于等距螺线模型，在其他模型中未必适用。

2、在盘旋过程中速度分析仅考虑了每个板凳把手始终垂直的理想因素，并未使用力学理论对其他非垂直因素分析，在其他因素下速度会发生改变，由于题目中所给条件不多，因此无法受力分析，结果可能不妥当。

3、题目中角度测算均为估算，轨迹方程可能存在误差。

模型改进与推广

1、在原有模型的基础上，增加实时反馈和调整机制，使模型能够根据现场情况动态调整运动轨迹和速度。例如，当遇到突发情况或场地限制时，模型能够迅速计算出最优的调整策略。

2、本文的模型不仅适用于舞龙队的运动规划，还可以推广到其他类似的长链式运动物体的轨迹规划和优化问题，如集会游行队伍、大型艺术表演等。

十二、参考文献

[1]姜行洲.几类微分方程边值方法的数值分析[D].哈尔滨工业大学.2021.

[2]薛均晓,张世文,陆亚飞,等.基于深度强化学习的无人机博弈路径规划[J/OL].郑州大学学报(理学版),1-7[2024-09-08].https://doi.org/10.13705/j.issn.1671-6841.2024033.

[3]张能俊.几何迭代法在拟合离散数据点中的应用[D].合肥工业大学.2021.

附录

表格目录：

问题一	**附录 1.1 龙头、龙身、龙尾在第 0、60、120、180、240、300s 时刻坐标** **附录 1.2 龙头、龙身、龙尾在第 0、60、120、180、240、300s 时刻速度**
问题二	**附录 1.3 龙头前把手、龙头后第 1、51、101、151、201 条龙身前把手和龙尾后把手坐标及速度**
问题四	**附录 1.4 第-100、−50、0、50、100s 时刻龙头前把手、龙头后第 1、51、101、151、201 条龙身前把手和龙尾后把手坐标** **附件 1.5 第 100、−50、0、50、100s 时刻龙头前把手、龙头后第 1、51、101、151、201 条龙身前把手和龙尾后把手速度**

代码目录：

问题一	**附录 2.1 问题一主程序 q_1.m**
问题二	**附录 2.2 问题二主程序 q_2.m**
问题三	**附录 2.3 问题三主程序 q_3.m**
问题四	**附录 2.4 问题四主程序 q_4.m**
问题五	**附录 2.5 问题五主程序 q_5.m**

附录 1.1 龙头、龙身、龙尾在第 0、60、120、180、240、300s 时刻坐标

	0 s	60 s	120 s
龙头 x (m)	8.80000	5.82284	-4.02466
龙头 y (m)	-3.44861	-5.74776	-6.34409
第 1 节龙身 x (m)	8.36629	7.46702	-1.38556
第 1 节龙身 y (m)	2.81900	-3.41869	-7.41805
第 51 节龙身 x (m)	-9.51202	-8.68114	-5.55254
第 51 节龙身 y (m)	1.38515	2.55714	6.36994
第 101 节龙身 x (m)	2.84106	5.64377	5.33948
第 101 节龙身 y (m)	-9.93878	-8.03144	-7.57321

第 151 节龙身 x (m)	10.8775	6.74561	2.44759
第 151 节龙身 y (m)	1.72701	8.08123	9.71223
第 201 节龙身 x (m)	4.67143	-6.53090	-10.6147
第 201 节龙身 y (m)	10.6739	9.08895	1.44863
龙尾（后）x (m)	-5.42725	7.27181	10.9808
龙尾（后）y (m)	-10.6141	-8.87363	0.742452

	180 s	240 s	300 s
龙头 x (m)	-3.07081	2.75470	4.29852
龙头 y (m)	6.04320	-5.27910	2.54776
第 1 节龙身 x (m)	-5.30500	4.91668	2.25592
第 1 节龙身 y (m)	4.27893	-3.43307	4.51469
第 51 节龙身 x (m)	2.84707	6.02738	-6.31167
第 51 节龙身 y (m)	7.26700	-3.75475	0.332010
第 101 节龙身 x (m)	1.89974	-4.89174	-6.27404
第 101 节龙身 y (m)	-8.47141	-6.40000	3.87937
第 151 节龙身 x (m)	1.04167	2.97547	7.03217
第 151 节龙身 y (m)	9.42044	8.39604	4.40705
第 201 节龙身 x (m)	-9.31559	-7.48656	-7.47269
第 201 节龙身 y (m)	-4.18374	-6.14441	-5.24315
龙尾（后）x (m)	7.44126	3.29836	1.82461
龙尾（后）y (m)	7.43444	9.44897	9.29310

附录 1.2 龙头、龙身、龙尾在第 0、60、120、180、240、300s 时刻速度

	0 s	60 s	120 s
龙头(m/s)	0.999472	0.998785	0.998559
第 1 节龙身(m/s)	0.999468	0.998775	0.998546
第 51 节龙身(m/s)	0.999434	0.998685	0.998423
第 101 节龙身(m/s)	0.999408	0.998622	0.998342
第 151 节龙身(m/s)	0.999389	0.998575	0.998284
第 201 节龙身(m/s)	0.999374	0.998539	0.998241
龙尾（后）(m/s)	0.999368	0.998526	0.998225

	180 s	240 s	300 s
龙头(m/s)	0.998232	0.997711	0.996757
第 1 节龙身(m/s)	0.998211	0.997677	0.996688
第 51 节龙身(m/s)	0.998035	0.997402	0.996197
第 101 节龙身(m/s)	0.997926	0.997249	0.995967
第 151 节龙身(m/s)	0.997853	0.997152	0.995833
第 201 节龙身(m/s)	0.997849	0.997085	0.995746
龙尾（后）(m/s)	0.997781	0.997062	0.995717

附录 1.3 龙头前把手、龙头后第 1、51、101、151、201 条龙身前把手和龙尾后把手坐标及速度

	横坐标 x(m)	纵坐标 y (m)	速度 (m/s)
龙头	1.26587	1.90368	0.999583
龙头后第 1 节龙身	-1.59215	1.79713	0.991150
龙头后第 51 节龙身	1.34460	4.30595	0.976701
龙头后第 101 节龙身	2.62241	-5.34363	0.974384
龙头后第 151 节龙身	3.97983	-5.83725	0.973485
龙头后第 201 节龙身	-6.80143	-4.25728	0.972993
龙尾后把手	1.02207	8.31421	0.972853

附录 1.4 第-100、−50、0、50、100s 时刻龙头前把手、龙头后第 1、51、101、151、201 条龙身前把手和龙尾后把手坐标

	-100 s	-50 s	0 s	50 s	100 s
龙头 x (m)	7.77803	6.30055	-2.71186	1.69608	-2.80018
龙头 y (m)	3.71716	2.8494	-3.59108	6.06871	7.67478
第 1 节龙身 x (m)	6.20927	4.70239	-0.06353	4.14004	0.03184
第 1 节龙身 y (m)	6.10852	5.22122	-4.67089	4.58323	8.07387
第 51 节龙身 x (m)	-10.6080	-2.69612	2.45996	-1.33577	2.41292
第 51 节龙身 y (m)	2.83149	-9.31651	-7.77815	-6.17452	3.82531
第 101 节龙身 x	-11.9228	10.6160	3.00849	-7.79144	-7.39828

(m)					
第 101 节龙身 y (m)	-4.80238	-5.1030	10.1085	4.88913	1.69152
第 151 节龙身 x (m)	-14.3510	13.2397	-7.00279	-4.28740	9.60116
第 151 节龙身 y (m)	-1.98099	-2.84834	10.3375	-10.5311	-3.17818
第 201 节龙身 x (m)	-11.9529	11.2268	-6.87284	0.685730	8.25066
第 201 节龙身 y (m)	10.5670	-10.1006	12.3826	-13.1712	8.88148
龙尾 x（后）(m)	-1.01106	0.189809	-1.93363	5.54377	-10.7787
龙尾 y（后）(m)	-16.5276	15.7206	-14.7131	12.7620	-7.10118

附件 1.6 第 100、−50、0、50、100s 时刻龙头前把手、龙头后第 1、51、101、151、201 条龙身前把手和龙尾后把手速度

	-100 s	-50 s	0 s	50 s	100 s
龙头(m/s)	0.999437	0.996472	0.987122	0.995799	0.999373
第 1 节龙身(m/s)	0.999352	0.996349	0.983383	0.995989	0.999483
第 51 节龙身(m/s)	0.998996	0.996866	0.984884	1.01172	1.01032
第 101 节龙身(m/s)	0.998834	0.997047	0.985175	1.01250	0.964641
第 151 节龙身(m/s)	0.998741	0.997140	0.985298	1.01274	0.964093
第 201 节龙身(m/s)	0.998680	0.997196	0.985367	1.01286	0.963879
龙尾（后）(m/s)	0.998660	0.997215	0.985388	1.01289	0.963821

附录 2.1

介绍：问题一主程序 q_1.m

```
clc, clear, close all

a = 0;
b = 1;
theta1 = 0;
theta2 = 32 * pi;
theta = linspace(theta1, theta2, 5000);
r = a + b * theta;
x = r.* cos(theta);
y = r.* sin(theta);
num = 224;
D_head = 3.41;
D_body = 2.20;
distance = 0.275;
k = 0.55;
v_head = 1.0;
T = 300;
dt = 1;

theta0 = 32 * pi ;
r0 = k * 16;

x_pos(1, 1) = r0 * cos(theta0);
y_pos(1, 1) = r0 * sin(theta0);

L = [D_head - 2 * distance; repmat(D_body - 2 * distance, num-1,
1)];

m_theta = theta0;
mid_r = r0;

for i = 2:num
    d_theta(i) = L(i-1) / mid_r;
    m_theta = m_theta + d_theta(i);
    mid_r = k / (2 * pi) * m_theta;
    x_pos(i, 1) = mid_r * cos(m_theta);
    y_pos(i, 1) = mid_r * sin(m_theta);
end

v(1:224, 1:301) = 1;

theta = theta0;
```

```
r = r0;

for j = dt:dt:T
    t = round(j / dt);

    theta_head = theta - v_head * dt / r;
    r_head = k / (2 * pi) * theta_head;
    x_pos(1, t+1) = r_head * cos(theta_head);
    y_pos(1, t+1) = r_head * sin(theta_head);
    theta = theta_head;
    r = r_head;
    m_theta = theta;
    mid_r = r;

    for i = 2:num
        d_theta(i) = L(i-1) / mid_r;
        m_theta = m_theta + d_theta(i);
        mid_r = k / (2 * pi) * m_theta;
        x_pos(i, t+1) = mid_r * cos(m_theta);
        y_pos(i, t+1) = mid_r * sin(m_theta);
    end

    if t > 0
        a = (x_pos(:, t+1) - x_pos(:, t)) / dt;
        b = (y_pos(:, t+1) - y_pos(:, t)) / dt;
        v(:, t+1) = sqrt(a.^2 + b.^2);
    end

end

times = 0:1:300;
x_pos_out = zeros(num, length(times));
y_pos_out = zeros(num, length(times));
v_put = zeros(num, length(times));

for i = 1:length(times)
    t_idx = round(times(i)/dt) + 1;
    x_pos_out(:, i) = x_pos(:, t_idx);
    y_pos_out(:, i) = y_pos(:, t_idx);
    v_put(:, i) = v(:, t_idx);
end
v_put(:,1)=( v_put(:,3)+1)/2;
coords_combined = zeros(length(times), 2 * num);
for t_idx = 1:length(times)
```

```
    coords_combined(t_idx, 1:2:end) = x_pos(:, t_idx);
    coords_combined(t_idx, 2:2:end) = y_pos(:, t_idx);
end

coords_combined = coords_combined';

% xlswrite('C:\Users\XXX\Desktop\A 题\附件\副
result1.xlsx',coords_combined,1,'B2');
% xlswrite('C:\Users\XXX\Desktop\A 题\附件\副
result1.xlsx',v_put,2,'B2');
```

附录 2.2

介绍：问题二主程序 q_2.m

```
clear,clc,close all;
l=0.55; % 螺距
k=l/2/pi;
L1=3.41;
D1=L1-0.275*2;
L2=2.2;
D2=L2-0.275*2;

theta=16*2*pi:-0.01:0*pi;
r=k*theta;
x=r.*cos(theta);
y=r.*sin(theta);

init_theta=@(t,theta)-1./(k*sqrt(1+theta.^2));
theta0=57.083706327857280; %300 秒开始出发
dt=0.1;
flag=0;%判断标志
step=0;

N=223;
X=zeros(N+1,3);
Y=zeros(N+1,3);
Th=zeros(N+1,3);
Th(1,3)=theta0;

while flag==0

    X(:,1)=X(:,3);
    Y(:,1)=Y(:,3);
    Th(:,1)=Th(:,3);
```

```
    ts=[0,dt/2,dt];
    [tt,theta]=ode45(init_theta,ts,Th(1,1));
    X1=k*theta.*cos(theta);
    Y1=k*theta.*sin(theta);
    step=step+1;
    X(1,:)=X1;
    Y(1,:)=Y1;
    Th(1,:)=theta;
    for j=2:length(tt)
        for i=2:N+1
            d=D1*(i<=2)+D2*(i>2);
            thetaij=s_th(l,X(i-1,j),Y(i-1,j),Th(i-1,j),d);
            Th(i,j)=thetaij;
            X(i,j)=k*thetaij*cos(thetaij);
            Y(i,j)=k*thetaij*sin(thetaij);
        end
    end

    for i=1:N
        x_1=X(i,2);x_2=X(i+1,2);
        y_1=Y(i,2);y_2=Y(i+1,2); % 确定两个孔的坐标
        theta_1=Th(i,2);
        theta_2=Th(i+1,2);  % 两个孔的角度

        index1=find((theta_1+2*pi-Th(:,2))>0); % 从外面开始
        index1=index1(end-2:end);
        index2=find(Th(:,2)-(theta_2+2*pi)>0);
        if isempty(index2)
            break; % 到达了盘入口
        else
            index2=index2(1:min(3,length(index2)));
        end
        idx_i=index1(1):index2(end);
        n=8;m=16;
        for jj=1:length(idx_i)-1
            X2_1=[X(idx_i(jj),2);Y(idx_i(jj),2)];
            X2_2=[X(idx_i(jj+1),2);Y(idx_i(jj+1),2)];

pd=find_obs(L1*(i<=1)+L2*(i>1),[x_1;y_1],[x_2;y_2],L2,X2_1,X2_2,n,m
);
            if ~isempty(pd)
                flag=1;
                break;
            end
```

```
        end
        if flag==1
            break;
        end
    end

    dt*step+300

end

V=-k*sqrt(1+Th(:,end).^2).*(Th(:,end)-Th(:,end-1))/(dt/2);
result = [X(:,3) Y(:,3) V];
disp("碰撞的时间是: ");
T_off = 300 + (step)*dt
% xlswrite('C:\Users\XXX\Desktop\A 题\附件
\result12.xlsx',result,1,'B2');

function flag=find_obs(L1,X1_1,X1_2,L2,X2_1,X2_2,n,m)
X1_cen=(X1_1+X1_2)/2;
X2_cen=(X2_1+X2_2)/2;
k1=(X1_1(2)-X1_2(2))/(X1_1(1)-X1_2(1)); % 第一个长方形的斜率
k1_=-1/k1;
k2=(X2_1(2)-X2_2(2))/(X2_1(1)-X2_2(1)); % 第二个长方形的斜率
k2_=-1/k2;

A=[k1_ -1
   k2_ -1];
P=A\[k1_*X1_cen(1)-X1_cen(2);k2_*X2_cen(1)-X2_cen(2)];
v1=X1_cen-P;
v2=X2_cen-P; % 两个长方形中心点与新原点连线矢量
theta1=angle(v1(1)+1i*v1(2));
theta2=angle(v2(1)+1i*v2(2));
delta_theta=theta1-theta2;
d1=norm(v1);
d2=norm(v2);

T=[cos(delta_theta) -sin(delta_theta)
    sin(delta_theta) cos(delta_theta)]; % 旋转矩阵
[X,Y]=meshgrid(linspace(d1-0.15,d1+0.15,n),linspace(0-
L1/2,0+L1/2,m));
X_Y=T*[X(:) Y(:)]';
flag=find(abs(X_Y(1,:)-d2)<30e-2/2 & abs(X_Y(2,:)-0)<L2/2);
end
```

```
function theta=s_th(l,x1,y1,theta1,d)
k=l/2/pi;
fun=@(theta)(k*theta.*cos(theta)-x1).^2+(k*theta.*sin(theta)-
y1).^2-d^2;
q=0.01;
options = optimoptions('fsolve','Display','off');
theta=fsolve(fun,theta1+q,options); % 计算角度
while abs(k*theta-k*theta1)>l/2 || theta<=theta1
    q=q+0.1;
    theta=fsolve(fun,theta+q,options);
end
    end
```

附录 2.3

介绍：问题三主程序 q_3.m

```
clc, clear, close all;

r0 = 8.8;
turn_r = 4.5;
v_head = 1;
d = 0.45;

lb = 0.45;
ub = 1;
options = optimoptions('ga', 'Display', 'iter', 'PopulationSize',
100, 'MaxGenerations', 10000, 'PlotFcn', @gaplotbestf);

[p_opt, fval] = ga(@(p) objective_function(p, r0, turn_r, v_head),
1, [], [], [], [], lb, ub, ...
                @(p) collision_constraints(p, r0, d, turn_r),
options);

fprintf('最优螺距 p = %.4f\n', p_opt);

t_final = time_eq(p_opt, r0, turn_r, v_head);
fprintf('盘入调头空间所需时间 t_turn = %.4f 秒\n', t_final);

theta_turn = theta_turn_eq(p_opt, r0, turn_r);
theta_vals = linspace(0, theta_turn, 1000);
r_vals = r0 + (p_opt / (2 * pi)) * theta_vals;
x_vals = r_vals .* cos(theta_vals);
y_vals = r_vals .* sin(theta_vals);
```

```
figure;
plot(x_vals, y_vals, 'b', 'LineWidth', 2);
hold on;
viscircles([0, 0], turn_r, 'LineStyle', '--', 'LineWidth', 2,
'Color', 'r');
title('螺旋曲线与调头空间');
xlabel('x （米）');
ylabel('y （米）');
axis equal;
legend('螺旋轨迹', '调头空间边界');
grid on;

function theta_turn = theta_turn_eq(p, r0, turn_radius)
    theta_turn = (2 * pi * (turn_radius - r0)) / p;
end

function t_turn = time_eq(p, r0, turn_radius, v_head)
    theta_turn = theta_turn_eq(p, r0, turn_radius);
    r_theta_turn = r0 + (p / (2 * pi)) * theta_turn;
    t_turn =  theta_turn* r_theta_turn / v_head;  % 计算时间
end

function [c, ceq] = collision_constraints(p, r0, d, turn_r)
    theta_turn = theta_turn_eq(p, r0, turn_r);
    theta_vals = linspace(0, theta_turn, 1000);
    r_vals = r0 + (p / (2 * pi)) * theta_vals;
    x_vals = r_vals .* cos(theta_vals);
    y_vals = r_vals .* sin(theta_vals);

    distances = sqrt(diff(x_vals).^2 + diff(y_vals).^2);
    c = d - max(distances);
    ceq = [];
end

function error = objective_function(p, r0, turn_radius, v_head)
    t_turn = time_eq(p, r0, turn_radius, v_head);  % 优化的目标是最小化
时间
    error = -t_turn;  % 返回时间作为优化目标
end
```

附录 2.4

介绍：问题四主程序 q_4.m

```
clc,clear, close all;

l = 1.7;
k = l / 2 / pi;
L1 = 3.41;
d1 = L1 - 0.275 * 2;
L2 = 2.2;
d2 = L2 - 0.275 * 2;
v = 1;

t = 5 * 2 * pi : -0.01 : 0 * pi;
r = k * t;
x = r .* cos(t);
y = r .* sin(t);

t = t - pi;%反向螺旋线
r2 = k * (t + pi);
x2 = r2 .* cos(t);
y2 = r2 .* sin(t);

R = 4.5; %调头边界圆
x_turn = R * cos(t);
y_turn = R * sin(t);

t_in = R / k;
t_out = R / k - pi;%在一条直线上相差 pi
slope = (k * sin(t_in) + R * cos(t_in)) / (k * cos(t_in) - R *
sin(t_in));
t_max_1 = atan(-1 / slope) + pi;
angle_eq = atan(tan(t_in)) + pi - t_max_1;
rC1_C2 = R / cos(angle_eq);
rC2 = rC1_C2 / 3;
rC1 = rC2 * 2;
phi = 2 * angle_eq;%重叠角
SC1 = rC1 * (pi - phi);
SC2 = rC2 * (pi - phi);
t_min_1 = t_max_1 - SC1 / rC1;
t_min_2 = t_min_1 - pi;
t_max_2 = t_min_2 + SC2 / rC2;
x_C1 = R * cos(t_in) + rC1 * cos(t_max_1 - pi);
y_C1 = R * sin(t_in) + rC1 * sin(t_max_1 - pi);
```

```
x_C2 = R * cos(t_out) - rC2 * cos(t_max_2);
y_C2 = R * sin(t_out) - rC2 * sin(t_max_2);

init_theta = @(t, t_val) 1 ./ (k * sqrt(1 + t_val .^ 2));

t0 = t_in;
dt = 1; %设置步长
tspan = 0:dt:100;
[tt, t_vals] = ode45(init_theta, tspan, t0);
X1 = k * t_vals .* cos(t_vals);
Y1 = k * t_vals .* sin(t_vals);

tt_in = tt(end:-1:1);
X = zeros(224, 200 / dt + 1);
Y = zeros(224, 200 / dt + 1);
TH = zeros(224, 200 / dt + 1);
X(1, 1:length(X1)) = X1(end:-1:1);
Y(1, 1:length(Y1)) = Y1(end:-1:1);
TH(1, 1:length(t_vals)) = t_vals(end:-1:1);

tt_c1 = dt:dt:SC1;
t_C1 = -tt_c1 / rC1 + t_max_1;
TH(1, length(t_vals) + (1:length(tt_c1))) = t_C1;
X(1, length(X1) + (1:length(tt_c1))) = rC1 * cos(t_C1) + x_C1;
Y(1, length(Y1) + (1:length(tt_c1))) = rC1 * sin(t_C1) + y_C1;

tt_c2 = tt_c1(end) + dt:dt:SC1 + SC2;
t_C2 = (tt_c2 - SC1) / rC2 + t_min_1 - pi;
TH(1, length(t_vals) + length(t_C1) + (1:length(tt_c2))) = t_C2;
X(1, length(X1) + length(t_C1) + (1:length(tt_c2))) = rC2 *
cos(t_C2) + x_C2;
Y(1, length(Y1) + length(t_C1) + (1:length(tt_c2))) = rC2 *
sin(t_C2) + y_C2;

init_theta_out = @(t, t_val) 1 ./ (k * sqrt(1 + (t_val + pi) .^
2));
t0_out = t_out;
tspan_out = tt_c2(end) + dt:dt:100;
[tt, t_vals_out] = ode45(init_theta_out, tspan_out, t0_out);
X2 = k * (t_vals_out + pi) .* cos(t_vals_out);
Y2 = k * (t_vals_out + pi) .* sin(t_vals_out);
TH(1, length(t_vals) + length(t_C1) + length(tt_c2) + 1:end) =
t_vals_out;
X(1, length(t_vals) + length(t_C1) + length(tt_c2) + 1:end) = X2;
```

```
Y(1, length(t_vals) + length(t_C1) + length(tt_c2) + 1:end) = Y2;

position_data = [];
N = 223;
t_total = -100:dt:100;

%开始计算各个点的位置信息
for jj=(1:length(t_total))
    j=round((t_total(1)+100)/dt)+jj;
    if t_total(jj)<=0
        for i=2:N+1
            d=d1*(i<=2)+d2*(i>2);
            thetaij=s_th1(l,X(i-1,j),Y(i-1,j),TH(i-1,j),d);
            TH(i,j)=thetaij;
            X(i,j)=k*thetaij*cos(thetaij);
            Y(i,j)=k*thetaij*sin(thetaij);
        end
    elseif t_total(jj)>0 && t_total(jj)<=SC1
        flag=2;
        for i=2:N+1
            d=d1*(i<=2)+d2*(i>2);
            if flag==2
                [xi,yi,thetai,flag]=solve2(l,X(i-1,j),Y(i-1,j),TH(i-
1,j),d,rC1,x_C1,y_C1,t_max_1);
                TH(i,j)=thetai;
                X(i,j)=xi;
                Y(i,j)=yi;
            else
                thetaij=s_th1(l,X(i-1,j),Y(i-1,j),TH(i-1,j),d);
                TH(i,j)=thetaij;
                X(i,j)=k*thetaij*cos(thetaij);
                Y(i,j)=k*thetaij*sin(thetaij);
            end
        end
    elseif t_total(jj)>SC1 && t_total(jj)<=SC1+SC2
        flag=3;
        for i=2:N+1
            d=d1*(i<=2)+d2*(i>2);
            if flag==3
                [xi,yi,thetai,flag]=solve3(X(i-1,j),Y(i-1,j),TH(i-
1,j),d,rC1,x_C1,y_C1,rC2,x_C2,y_C2,t_min_2);
                TH(i,j)=thetai;
                X(i,j)=xi;
                Y(i,j)=yi;
```

```
            elseif flag==2
                [xi,yi,thetai,flag]=solve2(l,X(i-1,j),Y(i-1,j),TH(i-
1,j),d,rC1,x_C1,y_C1,t_max_1);
                TH(i,j)=thetai;
                X(i,j)=xi;
                Y(i,j)=yi;
            else
                thetaij=s_th1(l,X(i-1,j),Y(i-1,j),TH(i-1,j),d);
                TH(i,j)=thetaij;
                X(i,j)=k*thetaij*cos(thetaij);
                Y(i,j)=k*thetaij*sin(thetaij);
            end
        end
    else
        flag=4;
        for i=2:N+1
            d=d1*(i<=2)+d2*(i>2);
            if flag==4
                [xi,yi,thetai,flag]=solve4(l,X(i-1,j),Y(i-1,j),TH(i-
1,j),d,rC2,x_C2,y_C2,t_max_2);
                TH(i,j)=thetai;
                X(i,j)=xi;
                Y(i,j)=yi;
            elseif flag==3
                [xi,yi,thetai,flag]=solve3(X(i-1,j),Y(i-1,j),TH(i-
1,j),d,rC1,x_C1,y_C1,rC2,x_C2,y_C2,t_min_2);
                TH(i,j)=thetai;
                X(i,j)=xi;
                Y(i,j)=yi;
            elseif flag==2
                [xi,yi,thetai,flag]=solve2(l,X(i-1,j),Y(i-1,j),TH(i-
1,j),d,rC1,x_C1,y_C1,t_max_1);
                TH(i,j)=thetai;
                X(i,j)=xi;
                Y(i,j)=yi;
            else
                thetaij=s_th1(l,X(i-1,j),Y(i-1,j),TH(i-1,j),d);
                TH(i,j)=thetaij;
                X(i,j)=k*thetaij*cos(thetaij);
                Y(i,j)=k*thetaij*sin(thetaij);
            end
        end
    end
```

```
    xy_data = [X(:, jj)'; Y(:, jj)'];
    xy_data = reshape(xy_data, [], 1)';  % 交叉存放 x 和 y 坐标
    position_data = [position_data; xy_data];

end
Vx=zeros(size(X));
Vy=Vx;
Vx(:,1)=(X(:,2)-X(:,1))/dt; %向前差分
Vx(:,end)=(X(:,end)-X(:,end-1))/dt; % 向后差分
Vx(:,2:end-1)=(X(:,3:end)-X(:,1:end-2))/2/dt;
Vy(:,1)=(Y(:,2)-Y(:,1))/dt;
Vy(:,end)=(Y(:,end)-Y(:,end-1))/dt;
Vy(:,2:end-1)=(Y(:,3:end)-Y(:,1:end-2))/2/dt;
V=sqrt(Vx.^2+Vy.^2);
position = position_data';
% xlswrite('C:\Users\XXX\Desktop\A 题\附件
\result4.xlsx',position,1,'B2');
% xlswrite('C:\Users\XXX\Desktop\A 题\附件\result4.xlsx',V,2,'B2');

function [x, y, t, flag] = solve4(l, x1, y1, t1, d, rC2, x_c, y_c,
t_max)
    k = l / 2 / pi;
    t = s_th2(l, x1, y1, t1, d);
    if t >= 4.5 / k - pi
        flag = 4;
        x = k * (t + pi) * cos(t);
        y = k * (t + pi) * sin(t);
    else
        fun = @(t_val) (x_c + rC2 * cos(t_max - t_val) - x1).^2 +
(y_c + rC2 * sin(t_max - t_val) - y1).^2 - d^2;
        q = -0.1;
        options = optimoptions('fsolve', 'Display', 'off');
        delta_t = fsolve(fun, t_max + q, options);
        t = t_max - delta_t;
        flag = 3;
        x = x_c + rC2 * cos(t);
        y = y_c + rC2 * sin(t);
    end
end


function theta = s_th2(l, x1, y1, t1, d)
    k = l / 2 / pi;
```

```
    fun = @(t) (k * (t + pi) .* cos(t) - x1).^2 + (k * (t + pi) .*
sin(t) - y1).^2 - d^2;
    q = -0.1;
    options = optimoptions('fsolve', 'Display', 'off');
    theta = fsolve(fun, t1 + q, options);
    while theta >= t1 || abs(k * theta - k * t1) > l / 2
        q = q - 0.1;
        theta = fsolve(fun, theta + q, options);
    end
end

function [x, y, t, flag] = solve2(l, x1, y1, t1, d, rC1, x_c, y_c,
t_max)
    k = l / 2 / pi;
    delta_t = 2 * asin(d / 2 / rC1);
    if delta_t <= t_max - t1
        flag = 2;
        t = t1 + delta_t;
        x = x_c + rC1 * cos(t);
        y = y_c + rC1 * sin(t);
    else
        t = s_th1(l, x1, y1, 4.5 / k, d);
        flag = 1;
        x = k * t * cos(t);
        y = k * t * sin(t);
    end
end

function [x, y, t, flag] = solve3(x1, y1, t1, d, rC1, x_c1, y_c1,
rC2, x_c2, y_c2, t_min)
    delta_t = 2 * asin(d / 2 / rC2);
    if delta_t <= t1 - t_min
        flag = 3;
        t = t1 - delta_t;
        x = x_c2 + rC2 * cos(t);
        y = y_c2 + rC2 * sin(t);
    else
        di = sqrt((x1 - x_c1)^2 + (y1 - y_c1)^2);
        delta_t = acos((di^2 + rC1^2 - d^2) / 2 / di / rC1);
        t_C1_di = atan((y1 - y_c1) / (x1 - x_c1));
        t = t_C1_di + delta_t;
        flag = 2;
        x = x_c1 + rC1 * cos(t);
        y = y_c1 + rC1 * sin(t);
```

```
    end
end

function theta = s_th1(l, x1, y1, t1, d)
    k = l / 2 / pi;
    fun = @(t) (k * t .* cos(t) - x1).^2 + (k * t .* sin(t) -
y1).^2 - d^2;
    q = 0.01;
    options = optimoptions('fsolve', 'Display', 'off');
    theta = fsolve(fun, t1 + q, options);
    while theta <= t1 || abs(k * theta - k * t1) > l / 2
        q = q + 0.1;
        theta = fsolve(fun, theta + q, options);
    end
end
```

附录 2.5

介绍：问题五主程序 q_5.m

```
clc,clear,close all;
l=1.7;
k=l/2/pi;
L1=3.41;
d1=L1-0.275*2; % 龙头孔间距
L2=2.2;
d2=L2-0.275*2; % 其他凳子把手两个孔之间的距离
V0=1:0.01:2;
Vx=[];
Vy=[];
for v=1:length(V0) %

    R=4.5;
    t_in=R/k;
    t_out=R/k-pi;
    slope=(k*sin(t_in)+R*cos(t_in))/(k*cos(t_in)-R*sin(t_in));
    th_max_1=atan(-1/slope)+pi;
    th_eq=atan(tan(t_in))+pi-th_max_1; % 等腰三角形的底角
    rC1_C2=R/cos(th_eq);
    rC2=rC1_C2/3;
    rC1=rC2*2;
    phi=2*th_eq; % C2 的重叠
    SC1=rC1*(pi-phi); SC2=rC2*(pi-phi);
    th_min_1=th_max_1-SC1/rC1;
    th_min_2=th_min_1-pi;
```

```
theta_max_2=th_min_2+SC2/rC2;
x_C1=R*cos(t_in)+rC1*cos(th_max_1-pi);
y_C1=R*sin(t_in)+rC1*sin(th_max_1-pi); % 得到圆 C1 的圆心坐标

x_C2=R*cos(t_out)-rC2*cos(theta_max_2);
y_C2=R*sin(t_out)-rC2*sin(theta_max_2); % 得到圆 C2 的圆心坐标
 v0=V0(v);
dt=0.1; % 时间步长
T_all=(SC1+SC2)/v0;
T=0:dt:T_all;
X=zeros(224,length(T));
Y=zeros(224,length(T));
TH=zeros(224,length(T));
% 圆弧 C1 上
tt_c1=0:dt:SC1/v0;
theta_C1=-v0*tt_c1/rC1+th_max_1;
TH(1,(1:length(tt_c1)))=theta_C1;
X(1,(1:length(tt_c1)))=rC1*cos(theta_C1)+x_C1;
Y(1,(1:length(tt_c1)))=rC1*sin(theta_C1)+y_C1;
% 圆弧 C2 上
tt_c2=tt_c1(end)+dt:dt:(SC1+SC2)/v0;
theta_C2=v0*(tt_c2-SC1/v0)/rC2+th_min_1-pi;
TH(1,length(theta_C1)+(1:length(tt_c2)))=theta_C2;
X(1,length(theta_C1)+(1:length(tt_c2)))=rC2*cos(theta_C2)+x_C2;
Y(1,length(theta_C1)+(1:length(tt_c2)))=rC2*sin(theta_C2)+y_C2;

N=223;
t_total=0:dt:(SC1+SC2)/v0;
for idx=(1:length(t_total))
    j = idx;
    if t_total(idx)<0
        for i=2:N+1
            d=d1*(i<=2)+d2*(i>2);
            thetaij=s_th1(l,X(i-1,j),Y(i-1,j),TH(i-1,j),d);
            TH(i,j)=thetaij;
            X(i,j)=k*thetaij*cos(thetaij);
            Y(i,j)=k*thetaij*sin(thetaij);

        end
    elseif t_total(idx)>=0 && t_total(idx)<=SC1/v0 % 头节点始终位于
圆弧 C1 段
        flag=2;
        for i=2:N+1
            d=d1*(i<=2)+d2*(i>2);
```

```
            if flag==2 % 在圆弧 1 区域
                [xi,yi,thetai,flag]=solve2(l,X(i-1,j),Y(i-
1,j),TH(i-1,j),d,rC1,x_C1,y_C1,th_max_1);
                TH(i,j)=thetai;
                X(i,j)=xi;
                Y(i,j)=yi;
            else
                thetaij=s_th1(l,X(i-1,j),Y(i-1,j),TH(i-1,j),d);
                TH(i,j)=thetaij;
                X(i,j)=k*thetaij*cos(thetaij);
                Y(i,j)=k*thetaij*sin(thetaij);
            end
        end
    elseif t_total(idx)>SC1/v0 && t_total(idx)<=SC1/v0+SC2/v0
        flag=3; % 位于圆弧 C2 段
        for i=2:N+1
            d=d1*(i<=2)+d2*(i>2); %判断孔的位置
            if flag==3
                [xi,yi,thetai,flag]=solve3(X(i-1,j),Y(i-1,j),TH(i-
1,j),d,rC1,x_C1,y_C1,rC2,x_C2,y_C2,th_min_2);
                TH(i,j)=thetai;
                X(i,j)=xi;
                Y(i,j)=yi;
            elseif flag==2
                [xi,yi,thetai,flag]=solve2(l,X(i-1,j),Y(i-
1,j),TH(i-1,j),d,rC1,x_C1,y_C1,th_max_1);
                TH(i,j)=thetai;
                X(i,j)=xi;
                Y(i,j)=yi;
            else
                thetaij=s_th1(l,X(i-1,j),Y(i-1,j),TH(i-1,j),d);
                TH(i,j)=thetaij;
                X(i,j)=k*thetaij*cos(thetaij);
                Y(i,j)=k*thetaij*sin(thetaij);
            end
        end
    else
        flag=4; % ，头节点在盘出螺线
        for i=2:N+1
            d=d1*(i<=2)+d2*(i>2);
            if flag==4
                [xi,yi,thetai,flag]=solve4(l,X(i-1,j),Y(i-
1,j),TH(i-1,j),d,rC2,x_C2,y_C2,theta_max_2);
                TH(i,j)=thetai;
```

```
                X(i,j)=xi;
                Y(i,j)=yi;
            elseif flag==3
                [xi,yi,thetai,flag]=solve3(X(i-1,j),Y(i-1,j),TH(i-
1,j),d,rC1,x_C1,y_C1,rC2,x_C2,y_C2,th_min_2);
                TH(i,j)=thetai;
                X(i,j)=xi;
                Y(i,j)=yi;
            elseif flag==2
                [xi,yi,thetai,flag]=solve2(l,X(i-1,j),Y(i-
1,j),TH(i-1,j),d,rC1,x_C1,y_C1,th_max_1);
                TH(i,j)=thetai;
                X(i,j)=xi;
                Y(i,j)=yi;
            else
                thetaij=s_th1(l,X(i-1,j),Y(i-1,j),TH(i-1,j),d);
                TH(i,j)=thetaij;
                X(i,j)=k*thetaij*cos(thetaij);
                Y(i,j)=k*thetaij*sin(thetaij);
            end
        end
    end
end

Vx(:,1)=(X(:,2)-X(:,1))/dt;
Vx(:,end)=(X(:,end)-X(:,end-1))/dt;
Vx(:,2:end-1)=(X(:,3:end)-X(:,1:end-2))/dt/2;
Vy(:,1)=(Y(:,2)-Y(:,1))/dt;
Vy(:,end)=(Y(:,end)-Y(:,end-1))/dt;
Vy(:,2:end-1)=(Y(:,3:end)-Y(:,1:end-2))/dt/2;
V=sqrt(Vx.^2+Vy.^2);
% 速度是否超过 2 m/s
if any(V(:) > 2)
    disp(['速度超限，v0 = ' num2str(v0) ' 时有节点速度超过 2 m/s，停止
计算。']);
    break; % 停止
end
end


function theta=s_th1(l,x1,y1,theta1,d)
k=l/2/pi;
fun=@(theta)(k*theta.*cos(theta)-x1).^2+(k*theta.*sin(theta)-
y1).^2-d^2;% 利用距离约束
```

```
q=0.01;
options = optimoptions('fsolve','Display','off');
theta=fsolve(fun,theta1+q,options);
while theta<=theta1 || abs(k*theta-k*theta1)>l/2
    q=q+0.1;
    theta=fsolve(fun,theta+q,options); % 重新求一个角度
end
end

function theta=s_th2(l,x1,y1,theta1,d)
k=l/2/pi;
fun=@(theta)(k*(theta+pi).*cos(theta)-
x1).^2+(k*(theta+pi).*sin(theta)-y1).^2-d^2;
q=-0.1;
options = optimoptions('fsolve','Display','off');
theta=fsolve(fun,theta1+q,options);
while theta>=theta1 || abs(k*theta-k*theta1)>l/2
    q=q-0.1;
    theta=fsolve(fun,theta+q,options);
end
end


function
[x,y,theta,flag]=solve2(luoju,x1,y1,theta1,d,rC1,x_c,y_c,theta_max)

k=luoju/2/pi;
delta_theta=2*asin(d/2/rC1);
if delta_theta<=theta_max-theta1
    flag=2;
    theta=theta1+delta_theta;
    x=x_c+rC1*cos(theta);
    y=y_c+rC1*sin(theta);
else
    theta=s_th1(luoju,x1,y1,4.5/k,d);
    flag=1;
    x=k*theta*cos(theta);
    y=k*theta*sin(theta);
end
end
function
[x,y,theta,flag]=solve3(x1,y1,theta1,d,rC1,x_c1,y_c1,rC2,x_c2,y_c2,
t_min)
```

```
del_th=2*asin(d/2/rC2);
if del_th<=theta1-t_min
    flag=3;
    theta=theta1-del_th;
    x=x_c2+rC2*cos(theta);
    y=y_c2+rC2*sin(theta);
else
    di=sqrt((x1-x_c1)^2+(y1-y_c1)^2);
    del_th=acos((di^2+rC1^2-d^2)/2/di/rC1);
    theta_C1_di=atan((y1-y_c1)/(x1-x_c1));
    theta=theta_C1_di+del_th;
    flag=2;
    x=x_c1+rC1*cos(theta);
    y=y_c1+rC1*sin(theta);
end
end

function
[x,y,theta,flag]=solve4(l,x1,y1,theta1,d,rC2,x_c,y_c,t_max)

k=l/2/pi;
theta=s_th2(l,x1,y1,theta1,d);
if theta>=4.5/k-pi
    flag=4;
    x=k*(theta+pi)*cos(theta);
    y=k*(theta+pi)*sin(theta);
else
    fun=@(t)(x_c+rC2*cos(t_max-t)-x1).^2+(y_c+rC2*sin(t_max-t)-
y1).^2-d^2;
    q=-0.1;
    ops = optimoptions('fsolve','Display','off');
    delta_theta=fsolve(fun,t_max+q,ops);
    theta=t_max-delta_theta;
    flag=3;
    x=x_c+rC2*cos(theta);
    y=y_c+rC2*sin(theta);

end
    end
```